DEMOCRATIZING EFFICIENT SPIKING NEURAL NETWORK TRAINING WITH FORWARD-GATED ONLINE LEARNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Spiking Neural Networks (SNNs) promise energy-efficient computation, but their training remains a significant challenge. Current methods present a dilemma: high-performance algorithms like Backpropagation Through Time (BPTT) and its online variant, Online Training Through Time (OTTT), are compatible with standard deep learning frameworks but are bottlenecked by dense gradient computations that ignore the inherently sparse nature of SNNs. Conversely, methods that exploit this sparsity for massive efficiency gains often require custom, hardware-specific kernels, hindering widespread adoption and research. This work introduces Forward-Gated Online Training Through Time (FG-OTTT), a novel training algorithm that resolves this trade-off. FG-OTTT achieves true computational efficiency by generating a dynamic, predictive sparse mask during the forward pass. This mask is then used to fundamentally prune the backward pass before any expensive dense gradient computations occur. Crucially, our method is implemented entirely within standard frameworks like PyTorch by creating a custom autograd function, requiring no specialized kernels. We demonstrate FG-OTTT's effectiveness on the CIFAR-10 dataset, showing that it can reduce wall-clock training time by over 2.3x and backward-pass computational load by up to 90

1 INTRODUCTION

Spiking Neural Networks (SNNs) have emerged as a compelling, brain-inspired computing paradigm, offering the potential for significant energy savings on event-driven neuromorphic hardware. However, realizing this potential is contingent on developing effective and efficient training algorithms. The dominant approach to training deep SNNs involves adapting the backpropagation algorithm, typically through surrogate gradients to handle the non-differentiable nature of spiking neurons aut (b). This has led to powerful but computationally demanding methods like Backpropagation Through Time (BPTT) aut (a) and, more recently, its memory-efficient online approximation, Online Training Through Time (OTTT) Xiao et al. (2022). The primary challenge with these methods is a fundamental disconnect between the SNN's operational principle—sparse, event-driven communication—and the training process. During backpropagation, both BPTT and OTTT compute dense gradient tensors, performing costly matrix-matrix multiplications that scale poorly and fail to leverage the underlying sparsity of neural activity. This creates a significant bottleneck, making the training of deep SNNs on large datasets a resource-intensive endeavor. While some research has focused on creating genuinely sparse training algorithms that capitalize on event-driven dynamics Perez-Nieves & Goodman (2021), these solutions often require custom CUDA kernels. This specialization creates a high barrier to entry, limiting their adoption to research groups with the expertise to develop and maintain such low-level code, thereby fragmenting the research landscape. A naive solution, which we term post-hoc sparsification, involves computing the full dense gradient and then applying a sparse mask before the weight update. However, this fails to address the core problem, as the expensive dense computation has already been performed. The true challenge is to prevent the dense gradient from ever being computed. To this end, we propose Forward-Gated Online Training Through Time (FG-OTTT), a novel algorithm that achieves true computational efficiency while maintaining full compatibility with standard deep learning libraries. The core innovation of FG-OTTT is to compute a predictive binary mask for synaptic updates during the forward pass, using information that is locally available in time, such as Hebbian activity traces. This pre-computed mask is then used to prune the backward pass *before* the dense gradient calculation, effectively transforming the expensive dense matrix multiplications into much cheaper sparse ones. This is accomplished by wrapping standard layers in a custom `torch.autograd.Function`, a native framework feature that requires no specialized compilers or kernels. Our contributions can be summarized as follows:

- **Novel Algorithm:** We introduce Forward-Gated Online Training Through Time (FG-OTTT), a novel SNN training algorithm that uses a predictive forward-pass mask to perform a computationally sparse backward pass within standard deep learning frameworks.
- **Empirical Demonstration:** We demonstrate empirically that FG-OTTT provides a dramatic reduction in wall-clock training time and backward-pass FLOPs compared to dense online and offline methods, without requiring custom kernels.
- **Controllable Sparsity:** We introduce a target-driven regularization mechanism that gives practitioners direct control over the computational budget (sparsity), enabling a principled exploration of the accuracy-efficiency trade-off and facilitating hardware-aware model design.
- **State-of-the-Art Trade-off:** Through extensive experiments on CIFAR-10, we show that FG-OTTT establishes a new, superior Pareto front for accuracy versus computational cost, outperforming both dense OTTT and a naive post-hoc sparsification baseline.

2 Related Work

The training of Spiking Neural Networks has been approached from several distinct angles, each with its own trade-offs between biological plausibility, performance, and computational efficiency. Our work, FG-OTTT, builds upon and provides a practical solution to the limitations of several of these existing paradigms.

2.1 Gradient-Based SNN Training

The most successful methods for training deep SNNs for complex tasks rely on backpropagation. Backpropagation Through Time (BPTT) is the de facto standard, treating the SNN as a recurrent neural network unrolled over time. To overcome the non-differentiability of the spike generation process, these methods employ surrogate gradients Thiele et al. (2019); aut (b). While BPTT achieves state-of-the-art accuracy, its memory footprint scales linearly with the number of simulation time steps, making it infeasible for long temporal sequences. To address this, Online Training Through Time (OTTT) was proposed as a memory-efficient online alternative that approximates BPTT using eligibility traces Xiao et al. (2022). FG-OTTT is built directly upon the OTTT framework but critically addresses its primary limitation: the computational cost of its dense backward pass.

2.2 Sparse and Efficient Training

Recognizing the computational burden of dense backpropagation, several works have explored sparsity during training. Perez-Nieves et al. Perez-Nieves & Goodman (2021) introduced a sparse spiking gradient descent algorithm that leverages the spatiotemporal sparsity of SNNs to achieve significant speedups. However, its implementation relies on custom CUDA kernels, which limits its accessibility and integration with standard deep learning ecosystems. In contrast, FG-OTTT achieves computational sparsity during the backward pass *without* departing from standard, widely-used frameworks, thus democratizing efficient training. Other work on unstructured pruning for SNNs focuses on reducing model size and spiking activity for energy-efficient inference Chakraborty & Mukhopadhyay (2024), but does not directly tackle the computational bottleneck of the training backward pass, which is our primary focus.

2.3 Biologically Plausible Learning

A separate line of research explores learning rules that are more consistent with known biological mechanisms, moving away from the global error signals of backpropagation. These include methods based on Spike Timing-Dependent Plasticity (STDP) Goupy et al. (2024) and alternative credit assignment strategies like Target Propagation Meulemans et al. (2020); Ernoult et al. (2022). Some models also investigate how local neuromodulators could propagate credit information over time Liu et al. (2022) or use meta-learning to discover plasticity rules aut (c). While our method's Hebbian gating mechanism is biologically inspired, FG-OTTT's primary contribution is not in proposing a new theory of learning in the brain, but rather in engineering a computationally efficient and practical implementation of a gradient-based algorithm (OTTT) for conventional hardware. It bridges the gap between high-performance but inefficient dense methods and highly efficient but inaccessible custom sparse methods.

3 Background

To understand our proposed method, it is essential to first review the foundational concepts of Spiking Neural Networks and the specific training framework upon which we build.

3.1 Spiking Neural Networks (SNNs)

SNNs are a class of neural networks where communication between units (neurons) occurs via discrete events, or spikes, distributed over time. A common neuron model is the Leaky Integrate-and-Fire (LIF) neuron. The dynamics of an LIF neuron are governed by its membrane potential, $V(t)$, which integrates incoming spikes and decays over time. When $V(t)$ exceeds a certain threshold, the neuron fires a spike and its potential is reset. The non-differentiable nature of this spiking mechanism at the threshold poses a challenge for gradient-based training. This is typically circumvented by using a surrogate gradient, which replaces the true, zero-or-infinite derivative of the spike function with a smooth, continuous approximation during the backward pass aut (b).

3.2 Problem Setting: Online Training Through Time (OTTT)

Our work directly addresses the computational limitations of the Online Training Through Time (OTTT) algorithm Xiao et al. (2022). OTTT was developed as an efficient online approximation to the memory-intensive Backpropagation Through Time (BPTT) algorithm. In BPTT, the network is unrolled for T time steps, and the gradient of the total loss is computed with respect to the weights. This requires storing the entire computation graph for all T steps, which is prohibitive for long sequences.

OTTT avoids this by computing instantaneous gradients at each time step t and updating the weights in a forward-in-time manner. This online property is highly desirable, but introduces its own challenges, such as ensuring learning stability aut (d). The algorithm achieves its approximation by maintaining an eligibility trace for each synapse, which tracks the influence of past pre-synaptic activity. The weight update rule in OTTT at time step t for a weight w_{ij} connecting neuron i to neuron j can be expressed as a three-factor Hebbian-like rule: $\Delta w_{ij}(t) \propto \delta_j(t) \cdot e_{ij}(t)$, where $\delta_j(t)$ is the instantaneous error signal backpropagated to post-synaptic neuron j, and $e_{ij}(t)$ is the eligibility trace, which is a filtered history of pre-synaptic spikes from neuron i modulated by the surrogate gradient of the post-synaptic neuron j. This formulation allows for constant memory usage with respect to time, a significant advantage over BPTT. However, the primary computational bottleneck remains. Although the updates are performed online, the calculation of the error signal $\delta(t)$ and the gradient term $\delta_j(t) \cdot e_{ij}(t)$ for all synapses at each layer involves dense matrix multiplications. For a linear layer with input x and weights W, the backward pass computes $\texttt{grad\_input} = \texttt{grad\_output} \mathbin{@} W$ and $\texttt{grad\_weight} = \texttt{grad\_output.T} \mathbin{@} x$. These operations are computationally expensive and dominate the training time. Our goal is to reduce the Floating Point Operations (FLOPs) required for these calculations by exploiting the natural sparsity of SNNs, but without resorting to custom computational kernels.

4 Method

We introduce Forward-Gated Online Training Through Time (FG-OTTT), an algorithm designed to drastically reduce the computational cost of the backward pass in SNN training while remaining fully compatible with standard deep learning frameworks. FG-OTTT achieves this by predictively pruning the gradient computation graph during the forward pass.

4.1 Base Framework

Our method is built upon a standard implementation of OTTT Xiao et al. (2022). We utilize its online, forward-in-time learning paradigm, which leverages eligibility traces to approximate the BPTT gradient while maintaining constant memory complexity with respect to simulation time.

4.2 Forward Gating Mechanism

The core of FG-OTTT is a predictive gating mechanism that computes a binary mask M for the synaptic weights of each layer during the forward pass. This mask determines which synapses will be considered for gradient computation in the subsequent backward pass. The mask M is constructed from the logical OR of two components, a Hebbian gate and a stochastic gate.

- **Hebbian Gate (M_h):** Inspired by biological principles, this gate enables a synaptic update only when there is correlated pre- and post-synaptic activity. Specifically, for a synapse ij, the update is permitted if its pre-synaptic eligibility trace is active and the post-synaptic activity trace is also active. This condition is modulated by a set of learnable, per-neuron thresholds θ_j. All information required to compute this gate is available locally at the end of the forward pass for the current time step.
- **Stochastic Gate ($M_{\textbf{rand}}$):** To prevent synapses from becoming permanently inactive (a form of neuron death) and to encourage exploration, we introduce a simple and computationally cheap stochastic gate. At each training step, a small, fixed percentage of synaptic updates are enabled at random across the layer. This replaces more complex and computationally expensive 'salvage' mechanisms that might rely on gradient magnitudes, which are not available during the forward pass.

The final predictive mask is the element-wise union of these two gates: $M = M_h$ OR M_{rand}.

4.3 Framework-Native Sparse Backward Pass

This is our primary technical contribution. We implement the sparse backward pass using a custom `torch.autograd.Function`. This allows us to define bespoke forward and backward operations that are seamlessly integrated into the framework's automatic differentiation engine without requiring any custom CUDA kernels.

- **Forward Method:** The `forward` method of our custom function performs the standard layer operation (e.g., convolution or linear transformation). In addition, it computes the predictive mask M for the current time step and saves it, along with other necessary tensors like the input spikes, for use in the backward pass.
- **Backward Method:** The `backward` method receives the upstream gradient (`grad_output`). It then retrieves the saved mask M and uses it to perform sparse gradient calculations. The weight gradient is computed as $\texttt{grad\_weight} = (\texttt{grad\_output.T @ input\_spikes}) \cdot M$. Similarly, the error signal to be propagated to the previous layer is computed using a masked weight matrix: $\texttt{grad\_input} = \texttt{grad\_output @ } (W \cdot M)$. By performing element-wise multiplication with the binary mask M, we ensure that the expensive matrix-matrix multiplications are replaced by sparse operations, dramatically reducing the required FLOPs.

4.4 Annealed Target-Driven Regularization

To provide explicit control over the computational budget, we introduce a regularization term to the task loss that encourages the sparsity of the mask M to match a predefined target S_{target}. The total loss is: $L_{\text{total}} = L_{\text{task}} + \lambda(t) \cdot (\text{mean}(M) - S_{\text{target}})^2$. Here, $\text{mean}(M)$ represents the computational density (fraction of active synapses), S_{target} is the desired density, and $\lambda(t)$ is a regularization coefficient that is annealed over the course of training. This mechanism allows the learnable Hebbian thresholds θ_j to adapt, steering the network towards the desired level of computational sparsity while optimizing for the primary task.

5 Experimental Setup

To validate the effectiveness of FG-OTTT, we designed an experiment to rigorously benchmark its performance against key SNN training baselines on a standard static image classification task.

5.1 Objective and Hypothesis

The primary objective was to demonstrate that FG-OTTT achieves a superior trade-off between computational efficiency (wall-clock time and backward-pass FLOPs) and task accuracy. We hypothesized that FG-OTTT would significantly outperform dense online and offline methods by occupying a more favorable position on the accuracy-efficiency Pareto front. Specifically, we expected it to be substantially faster than dense OTTT and a naive post-hoc sparsification baseline, while maintaining competitive accuracy.

5.2 Dataset and Preprocessing

We used the CIFAR-10 dataset. Static images were converted into spike trains using Rate Encoding over a simulation duration of $T = 16$ time steps. Each pixel's intensity (normalized to [0, 1]) determined the probability of generating a Poisson spike at each time step. Standard data augmentation techniques, including random horizontal flips and random crops, were applied to the images before encoding.

5.3 Model Architecture

A consistent VGG-style SNN architecture was used for all experiments to ensure a fair comparison. The network comprised a sequence of convolutional layers followed by fully-connected layers, with Leaky Integrate-and-Fire (LIF) neurons and average pooling. The structure was: (128C3) $\rightarrow$ LIF $\rightarrow$ AvgPool2 $\rightarrow$ (256C3) $\rightarrow$ LIF $\rightarrow$ AvgPool2 $\rightarrow$ (512C3) $\rightarrow$ LIF $\rightarrow$ AvgPool2 $\rightarrow$ Flatten $\rightarrow$ (1024L) $\rightarrow$ LIF $\rightarrow$ (10L) $\rightarrow$ LIF. We used a smooth ATan function as the surrogate gradient for the LIF neuron's spike generation.

5.4 Baselines for Comparison

- **FG-OTTT (Proposed):** Our method was evaluated at three different target computational densities: $S_{\text{target}} \in \{0.1, 0.2, 0.3\}$.
- **Dense OTTT:** The standard online training method Xiao et al. (2022), serving as our primary accuracy and dense-computation baseline.
- **HS-OTTT (Post-hoc Sparsification):** A crucial baseline designed to demonstrate the importance of pre-computation sparsification. This method computes the full dense gradient in the backward pass and then applies a mask to sparsify it before the optimizer step. This isolates the computational benefit of FG-OTTT.
- **BPTT (Backpropagation Through Time):** The standard high-performance offline SNN training method, used as an accuracy ceiling baseline.

5.5 Metrics and Implementation Details

- **Performance:** Top-1 classification accuracy on the test set.
- **Efficiency:** Total wall-clock training time for 200 epochs and backward pass FLOPs per epoch, measured using the `fvcore` library. Peak GPU memory usage was also monitored.
- **Hardware and Software:** All experiments were run on a single NVIDIA A100 GPU using PyTorch and the SpikingJelly library.
- **Training Protocol:** We used the Adam optimizer with an initial learning rate of 1e-3 and a Cosine Annealing scheduler. The batch size was 128. The loss function was Cross-Entropy applied to the mean membrane potential of the output neurons over the T=16 steps.
- **Reliability:** Each experiment was repeated 5 times with different random seeds, and we report the mean and standard deviation for all metrics.

6 Results

Our experiments on the CIFAR-10 dataset confirm that FG-OTTT provides a significant advantage in computational efficiency with a controllable and graceful trade-off in accuracy. The main results comparing the methods are as follows. For BPTT, the test accuracy was 89.92 $\pm$ 0.21

6.1 Accuracy vs. Efficiency Trade-off

The dense baselines, BPTT and Dense-OTTT, set the accuracy ceiling at approximately 89-90

6.2 The Critical Role of Pre-Computation Sparsification

A key finding comes from comparing FG-OTTT_0.2 with the naive HS-OTTT_0.2 baseline. Despite both targeting the same level of gradient sparsity for the weight update, FG-OTTT is substantially more accurate (86.53

6.3 Controllable Computational Cost

The results show a direct correlation between the S_{target} hyperparameter and the measured backward pass GFLOPs. FG-OTTT_0.1, FG-OTTT_0.2, and FG-OTTT_0.3 use approximately 10

6.4 Limitations

While FG-OTTT demonstrates a strong performance profile, its effectiveness relies on the tuning of the annealing schedule for the regularization parameter $\lambda(t)$. An improperly tuned schedule could lead to suboptimal convergence. Furthermore, the simple stochastic gate, while computationally cheap, may not be as effective as a more informed exploration strategy in certain network architectures or tasks. Future work could explore more adaptive or learned gating mechanisms.

7 Conclusion

In this work, we addressed a critical dilemma in the training of Spiking Neural Networks: the trade-off between computational efficiency and accessibility. We introduced Forward-Gated Online Training Through Time (FG-OTTT), a novel training algorithm that achieves massive computational savings by pruning the backward pass using a predictive mask generated during the forward pass. Our key contribution is an implementation that works entirely within standard deep learning frameworks, eliminating the need for custom CUDA kernels and thereby democratizing efficient SNN research. Our experiments on CIFAR-10 demonstrated that FG-OTTT can reduce wall-clock training time by more than half and backward-pass FLOPs by up to 90

This work was generated by AIRAS (Tanaka et al., 2025).

References

Can the brain do backpropagation? — exact implementation of backpropagation in predictive coding networks. a.

How gradient estimator variance and bias impact learning in neural networks. b.

A meta-learning approach to (re)discover plasticity rules that carve a desired function into a neural network. c.

Online stabilization of spiking neural networks. d.

Biswadeep Chakraborty and Saibal Mukhopadhyay. Towards energy efficient spiking neural networks: An unstructured pruning framework. 2024.

Maxence Ernoult, Fabrice Normandin, Abhinav Moudgil, Sean Spinney, Eugene Belilovsky, Irina Rish, Blake Richards, and Yoshua Bengio. Towards scaling difference target propagation by learning backprop targets. 2022.

Gaspard Goupy, Pierre Tirilly, and Ioan Marius Bilasco. Neuronal competition groups with supervised stdp for spike-based classification. 2024.

Yuhan Helena Liu, Stephen Smith, Stefan Mihalas, Eric Shea-Brown, and Uygar Sümbül. Biologically-plausible backpropagation through arbitrary timespans via local neuromodulators. 2022.

Alexander Meulemans, Francesco S. Carzaniga, Johan A. K. Suykens, João Sacramento, and Benjamin F. Grewe. A theoretical framework for target propagation. 2020.

Nicolas Perez-Nieves and Dan F. M. Goodman. Sparse spiking gradient descent. *Advances in Neural Information Processing Systems (NeurIPS, 2021)*, 2021.

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL `https://github.com/airas-org/airas`.

Johannes Christian Thiele, Olivier Bichler, and Antoine Dupret. Spikegrad: An ann-equivalent computation model for implementing backpropagation with spikes. 2019.

Mingqing Xiao, Qingyan Meng, Zongpeng Zhang, Di He, and Zhouchen Lin. Online training through time for spiking neural networks. 2022.